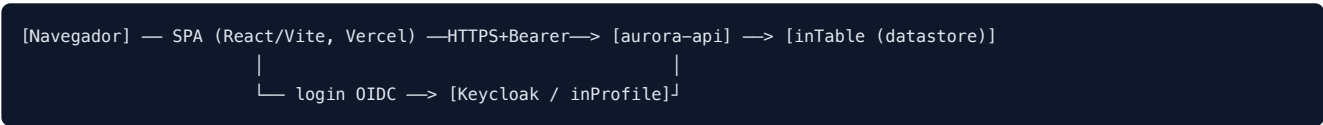


Documentação técnica da plataforma Aurora (saúde mental/bem-estar ocupacional).

Índice

- [Autenticação e RBAC](#)
- [Referência de API](#)
- [Comportamentos da plataforma](#)

Arquitetura (visão geral)



- **Frontend:** SPA (React + Vite) hospedada na **Vercel** (com Vercel Deployment Protection no sandbox).
- **Backend:** `aurora-api` (REST; formato de erro padrão NestJS: `{message,error,statusCode}`).
- **Integração de dados:** `inTable` (tabelas do tenant) — chave de API por tenant.
- **Autenticação:** **Keycloak** (realm `stg-inprofile`) via OIDC, com broker Microsoft (inbot).

Ambientes

Ambiente	URL
App (sandbox)	<code>https://sandbox-inbot-aurora.vercel.app</code>
API (sandbox)	<code>https://aurora-api-sandbox.inbot.com.br</code>
Keycloak	<code>https://keycloak.staging.e-auth.cloud/realms/stg-inprofile</code> (client <code>app-admin</code>)

Rotas do frontend

Rota	Tela
<code>/</code>	Home / Painel
<code>/colaboradores</code>	Gestão de Colaboradores
<code>/pedidos</code>	Pedidos de Protocolo
<code>/adesao</code>	Adesão e Engajamento
<code>/metricas-ia</code>	Métricas da IA
<code>/casos-graves</code>	Casos Graves
<code>/relatorios/individuais?protocolo=</code>	Relatórios Individuais (BAI/BHS/BDI/BSS)
<code>/relatorios/copsoq</code>	Relatório COPSQ
<code>/relatorios/consolidado</code>	Resultados Consolidados
<code>/auditoria</code>	Auditoria do Sistema
<code>/configuracoes</code>	Configurações

As rotas de relatório (`/relatorios/*`) carregam por navegação interna da SPA — ver [comportamentos da plataforma](#).

Autenticação e RBAC

Fluxo de autenticação

1. A SPA redireciona para o **Keycloak** (realms/stg-inprofile , client_id=app-admin).
2. A tela do Keycloak oferece login **federado via broker Microsoft ("inbot")**. Usuários federados (@inbot.com.br) **devem** usar o broker (não há senha local).
3. Após o SSO, a SPA obtém um **token Bearer (JWT)** e o renova em runtime (refresh silencioso via cookie de sessão Keycloak). O token **não é persistido** em localStorage.
4. Toda chamada à `aurora-api` envia o header **Authorization: Bearer <jwt>**.

Sandbox: a app está atrás de **Vercel Deployment Protection** — sessões automatizadas precisam também dos cookies Vercel (`_vercel_jwt`).

Contexto do usuário — GET /me

```
{
  "sub": "...",
  "email": "renato.paulino@inbot.com.br",
  "nome": "...",
  "perfil": "direcao",
  "tenantId": "...",
  "capabilities": ["colaborador.read", "..."],
  "needs_provisioning": false
}
```

Permissões (capabilities)

O acesso é controlado por **capabilities** entregues no `/me`. Cada endpoint exige a sua.

Capability	Habilita
colaborador.read/create/update/delete/import	CRUD + importação de colaboradores
relatorio.individual.read	Relatórios BAI/BHS/BDI/BSS
relatorio.copsoq.read	Relatório COPSOQ
relatorio.consolidado.read	Resultados Consolidados
relatorio.adesao.read	Adesão (/pedidos/lotes)
pedido.read/create	Pedidos (consulta/criação)
metricas-ia.read	Métricas da IA
caso-grave.read/update	Casos graves (consulta/tratativa)
kpi.read , kpi.casos-graves.read	Indicadores (dashboard/casos)
configuracao.read/visual.update/campos.update/provisioning.read	Configurações
auditoria.read	Auditoria

Referência de API

Base: `https://aurora-api-sandbox.inbot.com.br` · Auth: Authorization: Bearer <jwt> · Erros: { message, error, statusCode }.

Contexto

GET /me

Retorna o contexto/permissões do usuário. → { sub, email, nome, perfil, tenantId, capabilities[], needs_provisioning }

Colaboradores

GET /colaboradores

Query: page, limit (default 50), search (nome/e-mail), departamento, ativo. → { data: [{ id, nome, cpf, email, departamento, cargo, ativo, dataNascimento, telefone, genero, escolaridade, estadoCivil, ... }], page, limit, total }

POST /colaboradores

Body: { ativo, nome, cpf, email, dataNascimento (ISO yyyy-mm-dd), departamento, cargo, telefone } → 201 { id } Erros: 400 {message:["cpf inválido"]}, ["email must be an email"], ["dataNascimento ..."].

PUT /colaboradores/:id

Body completo (mesmos campos do POST) → 204. 404 se o id não existir. (PATCH e GET /colaboradores/:id → 404: não implementados.)

DELETE /colaboradores/:id

→ 204. 404 se não existir.

POST /colaboradores/import

multipart/form-data, campo file (CSV). Header CSV: nome,cpf,email,departamento,cargo,dataNascimento,telefone,escolaridade,estadoCivil,genero. → 201 { inserted, updated, errors } · 422 { message, erros: [{ linha, campo, motivo }] }.

Telas / Relatórios (somente leitura)

GET /screens/dashboard

→ { bemEstar, risco, balancaTrabalhoVida, casosGravesAbertos } (números; rótulos são calculados no front).

GET /screens/metricas-ia?periodo=7

→ { kpis: { interacoes, quantidadeUsuarios }, serie: [{ name, interacoes }] } (N pontos = dias do período).

GET /screens/relatorios/protocolo/{bai|bhs|bss|bdi}

→ { protocolo, periodo:{inicio,fim}, totalRespostas, mediaScore, distribuicao:{nome,quantidade}}, resultados: [{colaboradorId,nome,email,score,nivel,genero,faixaEtaria,escolaridade,areaTrabalho}], distribuicoesDemo:{...}, casosGravesPorDemo:{...}, perguntas:[{numero,descricao,mediaValor,nivelLabel}] } **BHS** usa a chave perguntasBhs: [{numero,descricao,percentualCritico,severidade}] .

GET /screens/relatorios/protocolo/copsoq

→ { periodo, totalJornadas, kpis:{ bemEstar:{valor,label}, risco:{valor,label}, balanca:{valor} }, distribuicoes:{...}, pontosFortes:[{dimensao,media,label,numQuestoes}], pontosAtencao:[...], riscoPorGrupos:{...} }

GET /screens/relatorios/consolidado

→ { periodo:{inicio,fim}, taxaParticipacao:{participantes,total,percentual}, kpis:{bemEstar,risco,balanca}, distribuicoes:{niveisBai,niveisBhs}, casosGraves:[{nome,email,protocolos[]}], resultados:[{colaboradorId,nome,email,linhas[]}] } (requisição sem ?periodo=.)

Casos Graves

GET /screens/casos-graves?periodo=30

periodo: 7|30 (ou histórico). → { kpis:{total,abertos,emAndamento,concluidos,arquivados}, casos:[{id,colaborador,colaboradorId,tipo,nivel,detalhe,status,responsavel,dataAbertura,dataAtualizacao,score,comentarios[],timeline[]}], agrupados:[...]} }

PATCH /casos-graves/:id/status

Body: { status } (Aberto|Em Andamento|Concluído|Arquivado) → 200 { id, status }. 422 {error:"Transição de status inválida", de, para} (máquina de estados).

POST /casos-graves/:id/comentarios

Body: { texto, autorNome, responsavelId } → 201 { id, casoGraveId, tipo:"comment", texto, autorNome, ... }.

Pedidos

GET /pedidos

Query: page, limit. → { data:[{id,protocolo,colaboradorEmail,status,criadoEm,atendidoEm,entrevistaId}], page, limit, total }

GET /pedidos/lotos → [{ protocolo, data, total, atendidos, percentual }]

GET /pedidos/adesao → [{ protocolo, total, atendidos, percentual }]

POST /pedidos (individual)

Body: { colaboradorId, protocolo } → 201 · 409 {message:"Já existe um pedido aberto de <P> para <colab>"}.

POST /pedidos/lote (em lote)

Body: { protocolo } → cria pedidos para **todos os colaboradores ativos** (ignora quem já tem pedido aberto).

Configurações

GET /configuracoes

→ { botId, syncEnabled, intableApiKeyConfigured, visual:{logoBase64,corPrincipal}, datasources:[...], departamentos:[...], cargos:[...]} }

GET /configuracoes/campos/uso → { departamentos:{<nome>:<count>}, cargos:{<nome>:<count>} }

GET /configuracoes/provisioning/status → { items:[{id,status,actionable}], tables:[{tableName,exists,active,rowCount,lastUpdated}] }

PATCH /configuracoes/visual — Body { logoBase64, corPrincipal } → 200

PATCH /configuracoes/departamentos — Body { departamentos:[] } → 200

PATCH /configuracoes/cargos — Body { cargos:[] } → 200

Auditoria

GET /auditoria

Retorna a lista de eventos { "Data e Hora", "Usuário", "Ação", "Recurso", "Decisão" } (requer auditoria.read).

Comportamentos da plataforma

Notas técnicas úteis para quem integra com a `aurora-api` ou automatiza a SPA.

Consistência / dados

- **Escritas assíncronas:** após `POST /colaboradores` (201), o registro pode levar alguns instantes para ficar consultável/operável por id. Planeje `retry/polling` em integrações server-to-server.
- **Paginação:** a API de colaboradores aceita `?page=&limit=` (default `limit=50`). Use os parâmetros para percorrer toda a base.

Relatórios

- **Navegação client-side:** as telas de relatório (`/relatorios/*`) carregam por navegação interna da SPA; nos relatórios individuais a troca de protocolo é feita pelas **abas** (BAI/BHS/BDI/BSS).
- **Contrato BHS:** o protocolo BHS retorna a chave `perguntasBhs: [{numero, descricao, percentualCritico, severidade}]` (os demais usam `perguntas`).

Validações

- **CPF:** a API valida o dígito verificador no cadastro; a importação CSV rejeita linhas com CPF inválido retornando `422` com o detalhe por linha.

Autenticação

- O token Bearer **não é persistido**; a renovação ocorre por refresh silencioso (cookie de sessão Keycloak). Integrações server-to-server devem usar fluxo próprio (service account / direct grant), não o `storageState` do browser.

Cobertura de testes (referência)

Suíte E2E + contratos de API em `tests/` (Playwright). A camada de API cobre relatórios, RBAC e CRUD de colaboradores/configurações/pedidos/tratativa. As telas rodaram em modo serial.